



1 Types & literals	
int i = 42;	32-bit integer
long n = 9_000_000L;	_ is a digit separator
double d = 3.14;	binary floating point
decimal m = 19.99m;	exact — the type for money
bool ok = true;	true or false
char c = 'A';	one UTF-16 unit
int h = 0x2A;	hex literal, 42
int b = 0b101010;	binary literal, 42
var x = new List<int>();	inferred at compile time
int n = default;	0, null or a zeroed struct
nint p = 42;	native-sized integer
const double Pi = 3.14159;	compile-time constant
readonly DateTime _t;	assignable only in a ctor
public required string Name { get; init; }	the initializer must set it · C# 11

2 Strings	
(\$"{greeting} {name}!")	interpolation · C# 6
@"C:\Users\Docs"	verbatim: escapes off
"""{ "a": 1 }"""	raw literal, no escaping · C# 11
\$\$\$""{ "n": "{x}" }"""	\$\$ makes {{ }} the hole · C# 11
var u8 = "Hi"u8;	UTF-8 literal, typed ReadOnlySpan<byte> · C# 11
"\e[31mError\e[0m"	\e is the ESC escape · C# 13
sb.Append(\$"Item {i}, ");	StringBuilder for loops
StringComparison.Ordinal	always pick a comparison
string.IsNullOrEmpty(s)	null or zero length

3 Null handling	
int? age = null;	nullable value type
string? name = null;	nullable reference type
customer?.Name?.Length	the chain short-circuits
userName ?? "Anonymous"	fallback when null
userName ??= "Anonymous";	assign only if null · C# 8
x is null	null test as a pattern
x is not null	ignores an overloaded ==
customer?.Order = order;	right side runs only if non-null · C# 14
customer?.Balance += 10;	compound too, not ++ · C# 14

4 Control flow	
if (a) X(); else Y();	two-way branch
else if (b) Z();	chain more conditions
age >= 18 ? "Adult" : "Kid"	ternary conditional
switch (day) { case DayOfWeek.Sunday: Weekend(); break; default: Weekday(); break; }	every section ends with break or return
string Kind(object x) => x switch { Circle => "Circle", null => "No shape", _ => "Other" };	switch expression · C# 8
Circle c when c.R > 10	when guards one arm

5 Loops	
for (int i = 0; i < 10; i++)	known iteration count
foreach (string n in names)	any IEnumerable
names.Index()	(index, item) tuples · .NET 9
foreach (var (k, v) in dict)	deconstruct as you go · C# 7
while (cond) { }	may run zero times
do { } while (cond);	runs at least once
continue;	skip to next iteration
break;	leave the loop
yield return item;	lazy iterator method

6 Pattern matching	
value is string text	type pattern, binds text
p is { Age: >= 30 }	property pattern
o is { Cust: { Name: "A" } }	nested property pattern
x is > 20 and < 30	and combinator · C# 9
s is "a" or "b"	or and not too · C# 9
(x, y) is (> 0, > 0)	tuple pattern
shape is Circle(var r)	positional, on a record
arr is [var a, .., var z]	list pattern with a slice · C# 11
arr is [> 0, > 0, ..]	first two positive
var (name, _) = person;	_ discards a value

7 Methods & parameters	
public int Add(int a, int b) => a + b;	expression-bodied · C# 6
void Greet(string hi = "Hi")	optional parameter
Greet(hi: "Yo");	named argument
void Bump(ref int n)	caller sees the change
bool Try(out int result)	assign before returning
double D(in Point p)	read-only reference · C# 7.2
void L(ref readonly Vec3 v)	caller passes a variable · C# 12
int Sum(params int[] n)	variable-length args
params ReadOnlySpan<int> n	over spans, no array alloc · C# 13

8 Lambdas & local functions	
int Outer(int n) { int Twice(int x) => x * 2; return Twice(n); }	local function · C# 7
Func<int, int> f = x => x;	lambda into a delegate
Action<string> log = Write;	Action returns void
(x, y = 10) => x + y	default lambda parameter · C# 12

9 Interfaces & generics	
public interface ILogger { void Log(string msg); void LogError(string m) => Log(\$"! {m}"); }	default interface implementation · C# 8
class Repo<T> where T : Emp	generic constraint
where T : struct	non-nullable value type
where T : class	reference type
where T : notnull	never null · C# 8
where T : new()	parameterless ctor; must be last
where T : allows ref struct	T may be Span<T> · C# 13
static T Sum<T>(List<T> xs) where T : INumber<T> { T total = T.Zero; foreach (T x in xs) total += x; return total; }	generic math · C# 11
static abstract T Create();	static abstract member · C# 11

10 Extension members	
public static bool IsBlank(this string s) => s.Trim().Length == 0;	classic extension method — still valid
extension<T>(IEnumerable<T> src) { public bool IsEmpty => !src.Any(); }	extension property; the block sits in a static class · C# 14
extension<T>(List<T>) { public static List<T> Empty => []; }	receiver type only: static members and operators · C# 14

11 Exceptions	
try { Risky(); } catch (IOException ex) { Log(ex); throw; } finally { Cleanup(); }	finally always runs
catch (Ex e) when (IsBad(e))	exception filter · C# 6
throw;	keeps the original stack trace
throw ex;	resets it — rarely wanted
throw new ArgumentException("Empty.", nameof(data));	always name the parameter
ArgumentNullException.ThrowIfNull(data);	guard clause · .NET 6
class NotFound(int id) : Exception(\$"No {id}") { public int Id => id; }	custom exception; no serialization ctor · .NET 8
int.TryParse(s, out int n)	use Try... for expected failures

12 Code organization	
namespace MyApp.Services;	file-scoped namespace · C# 10
global using System.Linq;	whole project · C# 10
using static System.Math;	then just Sqrt(x)
using Pt = (int X, int Y);	alias any type · C# 12
file class Validator { }	visible in this file only · C# 11
public	anywhere
internal	this assembly
protected	this type and derived types
private	this type only
protected internal	assembly OR derived
private protected	assembly AND derived
public partial string Conn { get; set; }	partial property · C# 13
#:package Spectre.Console #:sdk Microsoft.NET.Sdk.Web AnsiConsole.Write("Hello");	a whole app in one file: dotnet run app.cs · .NET 10

13 Classes, records & structs	
public int Age { get; set; }	auto property
public bool Ok => Age >= 18;	computed property
public int Id { get; init; }	set during initialization · C# 9
class Person(string name)	primary constructor · C# 12
record Person(string Name);	value equality, Deconstruct · C# 9
person with { Age = 31 }	copy, changing members
record struct Point(int X);	value-type record · C# 10
readonly record struct Rgb(byte R, byte G);	members become get/init
sealed class Cache { }	no further inheritance
abstract class Shape { }	must be derived from
static class Helpers { }	no instances

14 Collections	
int[] numbers = [1, 2, 3];	collection expression · C# 12
List<string> names = ["A"];	also sets, spans, interfaces
int[] more = [0, .. nums];	spread operator
Dictionary<string, int> d = new() { ["a"] = 1 };	no collection-expression form yet
d.TryGetValue(k, out int v)	lookup without throwing
ReadOnlySpan<int> s = array;	first-class spans, no copy · C# 14
numbers[^1]	last element · C# 8
numbers[1..^1]	range slice · C# 8
list[^1]	works, but List<T> has no range slicing
HashSet<int> set = [1, 2];	plus UnionWith, IntersectWith
queue.Enqueue(x);	Queue<T> is FIFO
stack.Push(x);	Stack<T> is LIFO

15 LINQ	
xs.Where(n => n > 5)	filter
xs.Select(p => p.Name)	project each element
xs.OrderBy(p => p.City)	sort; ThenBy for more keys
xs.GroupBy(p => p.City)	gives IGrouping
xs.First(n => n > 5)	also Any, All, Single
xs.Sum()	also Min, Max, Average
query.ToList()	LINQ is lazy until this
from n in xs where n > 5 select n * 2	query syntax

16 Newer LINQ operators	
xs.Chunk(3)	fixed-size batches · .NET 6
xs.MaxBy(p => p.Age)	also MinBy, DistinctBy · .NET 6
xs.CountBy(w => w)	count per key · .NET 9
xs.Index()	also AggregateBy · .NET 9
people.LeftJoin(orders, p => p.Id, o => o.CustomerId, (p, o) => p.Name)	keeps unmatched left rows · .NET 10
numbers.Shuffle()	also Sequence · .NET 10

17 Async	
async Task<string> Get()	Task<T> returns a value
async Task SaveAsync()	Task returns nothing
ValueTask<int> ReadAsync()	often already complete
await Task.WhenAll(t1, t2);	concurrent, wait for all
await Task.WhenAny(t1, t2);	first one to finish
Task.WhenEach(tasks)	handle each as it lands · .NET 9
await Task.Delay(10, token);	pass the token down
cts.CancelAfter(timeout);	catch OperationCanceled
await foreach (var x in Q())	IAsyncEnumerable · C# 8
.ConfigureAwait(false)	in libraries, don't capture
async void	event handlers only — cannot be awaited

18 New in C# 14 (.NET 10)	
extension(List<int> src)	properties, statics and operators
set => field = value;	field is the generated backing field
customer?.Order = value;	null-conditional assignment
nameof(List<>)	unbound generics, gives "List"
ReadOnlySpan<int> s = array;	first-class spans
(s, out n) => Try(s, out n)	lambda modifiers without types
partial Settings(string p);	partial constructors and events
public void operator +=(T x)	compound assignment, in place